

TD Sécurité Web

Objectif

Dans ce TD, vous allez analyser les vulnérabilités de sécurité de votre application web de recrutement. Vous choisirez 3 vulnérabilités parmi une liste proposée, puis vous illustrerez leur exploitation et implémenterez des solutions efficaces pour sécuriser l'application.

Questions

Vous devez :

1. Choisir 3 vulnérabilités parmi celles proposées ci-dessous.
2. Montrer par l'exemple comment ces vulnérabilités peuvent être exploitées dans l'application.
3. Implémenter une protection adaptée et vérifier son efficacité.
4. Pour les non-apprentis (SR10) :

Vous devez proposer au minimum trois cas d'utilisation des tests automatisés pour vérifier la conformité de l'application aux règles de sécurité. Vous pouvez réutiliser les vulnérabilités que vous avez sélectionnées dans la question 1 (attention le cas (a) est obligatoire). Chaque proposition doit être justifiée. Voici quelques cas possibles :

- a. Vérification de la gestion des sessions et des droits d'accès. Mettre en place des tests pour s'assurer qu'un utilisateur authentifié mais non autorisé ne peut pas accéder à des routes réservées à l'administration.

Exemple : un candidat connecté ne doit pas pouvoir accéder à /admin/panel.

- b. Simulation d'une attaque par force brute. Tester le comportement de l'application face à de multiples tentatives de connexion invalides. L'objectif est de vérifier qu'un mécanisme de limitation (compteur, délai, blocage temporaire, etc.) est bien en place.

Exemple : après 5 tentatives échouées, la connexion doit être bloquée pendant un certain temps.

- c. Vérification de la résistance à l'injection SQL. Automatiser des tests qui injectent des chaînes malveillantes dans les champs de saisie (comme ' OR 1=1--) afin de vérifier que les requêtes sont sécurisées (paramétrées, préparées, etc.).

Exemple : une tentative d'injection dans le champ "nom d'utilisateur" ne doit pas contourner l'authentification.

Vous pouvez ajouter d'autres propositions pertinentes, comme la détection de Cross-Site Scripting (XSS), le test des en-têtes HTTP de sécurité, ou la vérification de la non-exposition de données sensibles dans les réponses.

Liste des vulnérabilités proposées

Vulnérabilité	Description	Exemple	Ressources
Injection SQL	Code SQL malveillant injecté via des formulaires.	<code>`email = ' OR '1'='1`</code> dans le login.	https://owasp.org/www-community/attacks/SQL_Injection
Cross-Site Scripting (XSS)	Code JavaScript malveillant injecté dans les champs affichés.	<code>`<script>alert('XSS')</script>`</code> dans un titre.	https://owasp.org/www-community/attacks/xss/
Mots de passe non ou mal hachés	Stockage des mots de passe en clair dans la BDD. Une mauvaise implémentation de la fonction de hachage.	Fuite = lecture directe des mots de passe.	https://www.npmjs.com/package/bcrypt
Session Fixation	Réutilisation d'un identifiant de session imposé.	Attaquant force un session ID à la victime.	https://owasp.org/www-community/attacks/Session_fixation
CSRF	Exécution de requêtes à l'insu de l'utilisateur connecté.	Un lien externe déclenche une action sur le compte connecté.	https://www.npmjs.com/package/csrf
Contrôle d'accès	Accès sans vérification du rôle.	Un candidat accède à <code>`/admin/dashboard`</code> .	https://blog.logrocket.com/role-based-access-control-node-js/
Upload non sécurisé	Fichier malveillant envoyé (ex: .php, .exe).	Upload de <code>`shell.php`</code> .	https://github.com/expressjs/multer